Documentación del Flujo de Trabajo Transcriptómico: Bone (3 vs 24 meses)

1. Introducción

El presente documento resume el pipeline llevado a cabo hasta el alineamiento contra el genoma de referencia de secuencias de Bulk RNA-seq del estudio: "Ageing hallmarks exhibit organ-specific temporal signatures" (GSE132040).

- **Tejido:** Bone
- **Edades comparativas:** 3 vs 24 meses
- **Sexo:** Macho

2. Datos y Selección

Los metadatos fueron descargados de NCBI GEO. Utilizando un script de filtrado en Python (*ver Apéndice A.1*), se procesaron los datos para mantener únicamente las muestras que cumplieran los criterios fisiológicos requeridos, con un total de 7 accesiones SRRs: SRR9127063, SRR9127357, SRR9127308, SRR9127396, SRR9126859, SRR9127293, y SRR9126934.

3. Pipeline General

El flujo de trabajo se automatizó bajo los siguientes pasos:

1. **Descarga de datos:** Utilización de `sra-tools` (`prefetch` y `fasterq-dump`) para descarga de SRAs y conversión a fastq pareados.
2. **Quality Control (QC):** Empleo de `FastQC` + `MultiQC` a nivel crudo y procesado.
3. **Limpieza (Trimming):** Proceso altamente paralelizado (uso de `parallel`) con `fastp` (*ver Apéndice A.2*) para el perfilamiento biológico y técnico de las lecturas.
4. **Alineamiento referencial:** Ejecución automatizada con `STAR` y `HISAT2` en modalidades Paired-End (PE) y Single-End (SE) (*ver Apéndices A.3 - A.6*).

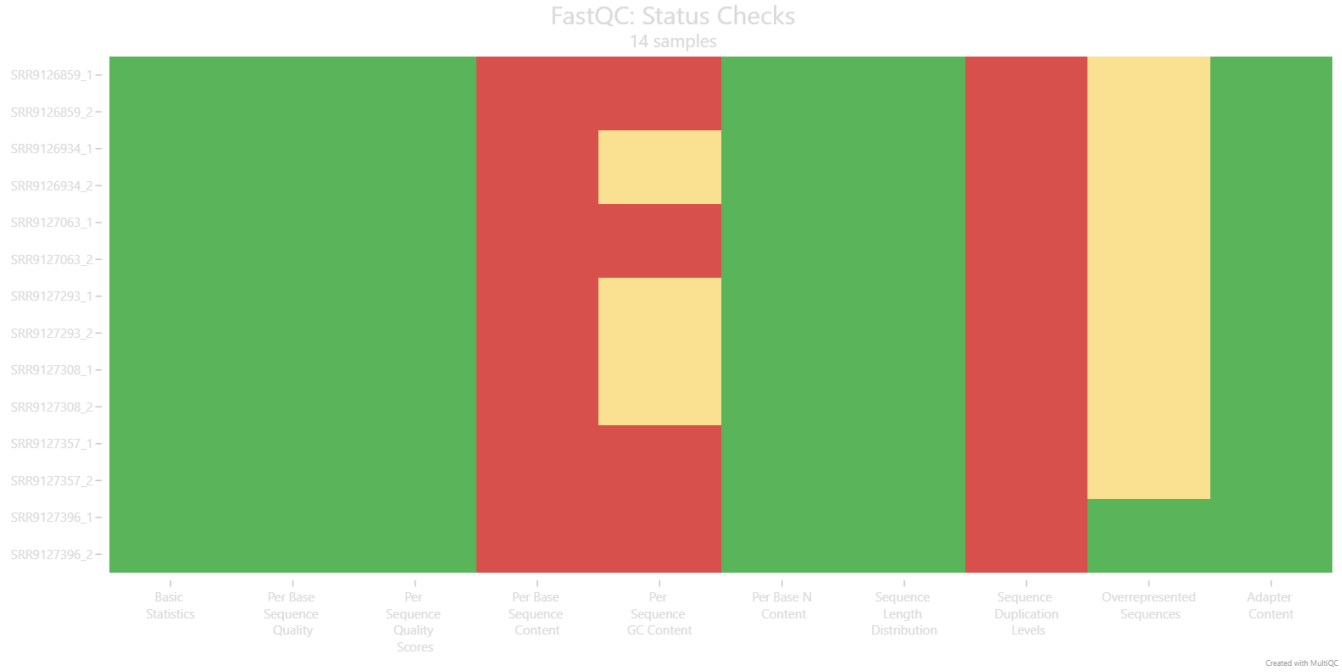
4. Control de Calidad

Hallazgos de Datos Crudos:

- **Secuenciador:** Illumina NovaSeq 6000 (100 bp).
- Presencia destacable de adaptadores *Nextera* y secuencias sobrerrepresentadas del oligonucleótido *Clontech SMARTer II A Oligonucleotide*.

- Se identificó ruido inicial sistemático (10-15 pb).

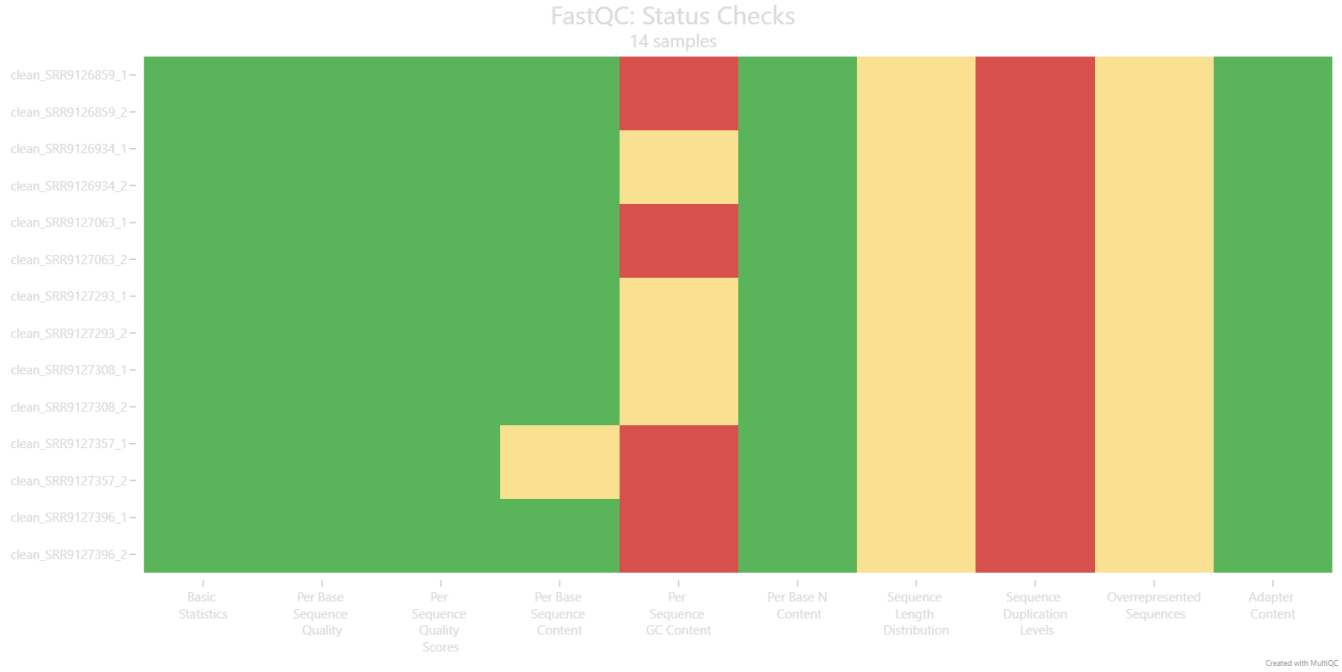
Heatmap del estado general de las muestras, generado por `multiqc`:



Hallazgos Post-Limpieza (Trimming):

A través del corte frontal, eliminación de adaptadores y detección de colas Poli-G, se restauró el equilibrio de calidad en las muestras. El estatus de "Per-Sequence-Content" transicionó de "FAIL" a "PASS" en la mayoría de las muestras.

Heatmap del estado general de las muestras, generado por `multiqc`:



5. Alineamiento - Diseño Comparativo

Se probaron distintos dos alineadores diferentes, cada uno probado con lecturas *paired-end* y *single-end*:

- **STAR vs HISAT2:** Diferentes alineadores orientados a variantes en splicing biológico.
- **SE vs PE:** Evaluación del mapeo tratando las lecturas como independientes *single-end* comparadas con las restricciones de orientación y distancia entre pares de las lecturas *paired-end*.

6. Resultados

Tiempos y Rendimiento Computacional

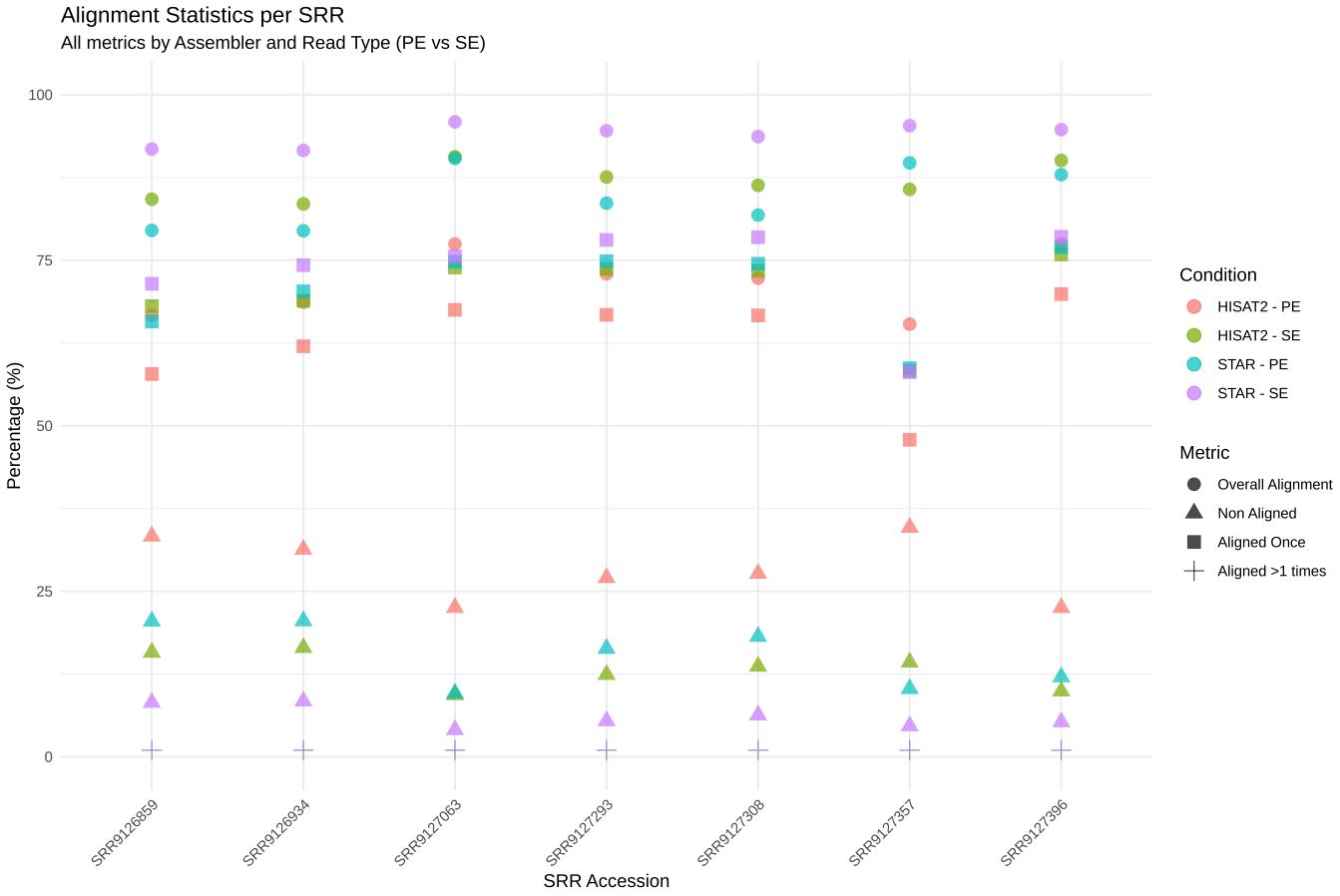
Herramienta / Modalidad	CPUs (uso promedio)	Tiempo Real Total
Limpieza (`fastp`)	~18.3	~3 minutos
STAR (Single-End)	~6.9	~35 minutos
STAR (Paired-End)	~6.0	~1h 49 minutos
HISAT2 (Single-End)	~3.9	~1h 38 minutos
HISAT2 (Paired-End)	~2.5	~15h 17 minutos

- STAR expuso una velocidad de procesamiento mayor frente a HISAT2

Nota: Los archivos de tiempo generados por `time` *se encuentran en sus respectivas carpetas bajo el nombre "time.txt"*

Estadísticas de Alineamiento

Los archivos de la validación fueron estructurados (**Rust**, *ver Apéndice B.1*) y graficados (**R**, *ver Apéndice B.2*) para el análisis visual.



7. Conclusiones

- **STAR sobre HISAT2:** STAR superó a HISAT2 al reportar menos lecturas *no alineadas* y un *aligned rate* consistentemente mejor (~5-10% superior en general), adicional a su beneficio en velocidad.
- **Single-End (SE) vs Paired-End (PE):** En todos los casos el modo SE aumentó aparentemente el porcentaje del éxito global. Al no imponerse forzosidad en concordancia de orientación y distancias fijas entre la hebra R1 y R2, SE puede "rescatar" alineamientos, no obstante, retiene menos rigor biológico. Para el conteo e isoformas downstream, los resultados obtenidos en PE seguirán siendo ideales.
- **Muestra Atípica:** Independientemente del pipeline usado, la muestra "SRR9127357" se presentó como un "outlier" técnico denotando los menores ratios de alineamiento único y picos en mapeos múltiples.

---

8. Reproducibilidad

Toda la construcción del proyecto y versionado del código documentado se encuentra alojado públicamente en GitHub:
[Hectorjlo/transcriptomica]

Apéndice de Scripts

A.1. Script de Filtrado de SRRs

Ref: `src/extractSRR.py`

```python
"""
Hardcoded python script to extract SRRs given some filters
"""
import pandas as pd

# Metadata file
META = "GSE132040_MACA_Bulk_metadata.csv"

# Filters
TISSUE = "Bone"
AGES = {3, 24}
SEX = "m"  # males

# Create a pd.DataFrame
df = pd.read_csv(META)

# Define the column names exactly as they appear in the metadata table.
# These names include spaces and punctuation, so we keep them centralized
# in variables to avoid typos and make future edits easier.
col_source = "source name"
col_age = "characteristics: age"
col_sex = "characteristics: sex"
col_srr = "raw file"

# Quick schema validation:
# check that all required columns are present before any filtering logic runs.
# If a required column is missing, stop early with a clear error message.
missing = [c for c in [col_source, col_age, col_sex, col_srr] if c not in
df.columns]
if missing:
    raise SystemExit(f"Missing columns in the CSV: {missing}\nAvailable
columns: {df.columns.tolist()}")

# Cleanup and type normalization:
# 1) standardize sex values to lowercase trimmed strings
# 2) coerce age to numeric so set membership checks behave consistently
df[col_sex] = df[col_sex].astype(str).str.strip().str.lower()
df[col_age] = pd.to_numeric(df[col_age], errors="coerce")
```

```python
# Filtering criteria:
# 1) source name starts with "Bone_" to keep only the target tissue entries
# 2) age is one of the selected groups in AGES ({3, 24})
# 3) sex matches the selected value in SEX ("m")
mask = (
    df[col_source].astype(str).str.startswith(f"{TISSUE}_") &
    df[col_age].isin(AGES) &
    (df[col_sex] == SEX)
)

sub = df.loc[mask, [col_source, col_age, col_sex, col_srr]].copy()

# Sort the filtered subset for cleaner and reproducible output ordering.
sub = sub.sort_values([col_age, col_source, col_srr])

# Extract the SRR identifiers as clean strings, one identifier per row.
srrs = sub[col_srr].astype(str).str.strip()

# Print a concise report:
# 1) full filtered table for verification
# 2) SRR-only list for downstream command-line workflows
print("=== Summary (Hector: Bone, 3 vs 24, males) ===")
print(sub.to_string(index=False))
print("\n=== SRR (one per line) ===")
for s in srrs:
    print(s)

# Save SRR list to a plain text file for convenience and reuse in pipelines.
out_txt = "hector_bone_3_vs_24_srr.txt"
srrs.to_csv(out_txt, index=False, header=False)
print(f"\nSaved: {out_txt}  (SRRs list)")
```

A.3. Automate HISAT2 (Single-End)

Ref: `src/auto_hisat2.sh`

```bash
#!/usr/bin/env bash

# Runs hisat2 single_end in parallel for each read, harcoded code.
# This script expects two arguments, one that is the path in which the
# files will be search on and the second where output files will be generated
# Usage:
#   ./auto_hisat2.sh <input_dir> <output_dir>
#   ./auto_hisat2.sh ../data/trimmed_fastqs ../results/hisat2/single_end
```

```
# Same principle as `auto_fastp.sh`
# Create an array of the names that match: clean_*_1.fastq
mapfile -t files < <(find "$1" -maxdepth 1 -type f -name "clean_*_1.fastq" |
sort)

# Count how many read_files are
n_files=$(( ${#files[@]} ))


# Hardcoded function
run_hisat2() {
    # Arguments
    #   -x: Path of the index of the reference genome
    #   -U: Input of the fastq file of single_end reads, taken from the first
argument passed to the function ($1)
    #   -S: Path of the output SAM file ($2)
    #   --summary-file: Path of output summary file ($3)
    #   -k: Maximum number of alignments reported per read
    #   --no-unal: Don't report unaligned reads in the SAM file
    hisat2 -x
/export/space3/users/silvanac/transcriptomica_2026/indexes/mm39.gencode.M36.hi
sat/mm39.gencode.M36.hisat \
           -U "$1" \
           -S "$2" \
           --summary-file "$3" \
           -k 7 \
           --no-unal

}
# Export the function, needed for parallel to use it
export -f run_hisat2

# Take the second argument passed to the main call ($2) and remove a "/"
adding at the end a "/"
out_dir="${2%/}/"
# For each read_file do ... until i is less than the number of elements in the
array "files"
# starting with i = 0
for (( i=0; i<${#files[@]}; i+=1 )); do
# Build the core_name var, it takes the element "i" of the array "files"
# starting from the front of the string will eliminate all "*" until a "/" is
found,
# then again starting from the front will eliminate the first instance of
"clean_",
# finally starting from the back of the string will eliminate the ".fastq"
```

```
# e.g:
#   ../../../data/trimmed_fastqs/cleaned_SRR1111.fastq --> SRR1111
core_name="${files[$i]##*/}"
core_name="${core_name#clean_}"
core_name="${core_name%.fastq}"
# The echo will send 3 strings:
#   1st -> The fastp file of unpaired reads
#   2nd -> Name of the output SAM file
#   3rd -> Name of the summary file
# These strings are build in the echo
echo "${files[$i]} ${out_dir}output_${core_name}.sam
${out_dir}summary_of_${core_name}.txt"
done | parallel -j ${n_files} --colsep ' ' run_hisat2
```

A.4. Automate HISAT2 (Paired-End)

Ref: `src/auto_hisat2_pe.sh`

```
#!/usr/bin/env bash

# Runs hisat2 paired_end in parallel for each read, harcoded code.
# This script expects two arguments, one that is the path in which the
# files will be search on and the second where output files will be generated
# Usage:
#   ./auto_hisat2_pe.sh <input_dir> <output_dir>
#   ./auto_hisat2_pe.sh ../data/trimmed_fastqs ../results/hisat2/paired_end

# Same principle as `auto_hisat2.sh`
# Create an array of the names that match: clean_*.fastq
mapfile -t files < <(find "$1" -maxdepth 1 -type f -name "clean_*.fastq" |
sort)

# Count how many read_files are / 2, pe reads
n_files=$(( ${#files[@]} / 2 ))


# Hardcoded function
run_hisat2() {
    # Arguments
    #   -x: Path of the index of the reference genome
    #   -1: Input of forward reads, taken from the first argument passed to
the function ($1)
    #   -2: Input of reverse reads, taken from the second argument passed to
the function ($2)
    #   -S: Path of the output SAM file ($3)
```

```
    #   --summary-file: Path of output summary file ($4)
    #   -k: Maximum number of alignments reported per read
    #   --no-unal: Don't report unaligned reads in the SAM file
    #   --no-mixed: Don't report mixed alignments for paired reads
    #   --no-discordant: Don't report discordant alignments for paired reads
    hisat2 -x
/export/space3/users/silvanac/transcriptomica_2026/indexes/mm39.gencode.M36.hi
sat/mm39.gencode.M36.hisat \
           -1 "$1" \
           -2 "$2" \
           -S "$3" \
           --summary-file "$4" \
           -k 7 \
           --no-unal \
           --no-mixed \
           --no-discordant

}
# Export the function, needed for parallel to use it
export -f run_hisat2

# Take the second argument passed to the main call ($2) and remove a "/"
adding at the end a "/"
# For each read_file do ... until i is less than the number of elements in the
array "files"
# starting with i = 0
out_dir="${2%/}/"
for (( i=0; i<${#files[@]}; i+=2 )); do
# Build the core_name var, it takes the element "i" of the array "files"
# starting from the front of the string will eliminate all "*" until a "/" is
found,
# then again starting from the front will eliminate the first instance of
"clean_",
# finally starting from the back of the string will eliminate the "_1.fastq"
# e.g:
#   ../../../data/trimmed_fastqs/clean_SRR1111_1.fastq --> SRR1111
# The echo will send 4 strings:
#   1st -> The fastq file of forward reads
#   2nd -> The fastq file of reverse reads
#   3rd -> Name of the output SAM file
#   4th -> Name of the summary file
# These strings are build in the echo
core_name="${files[$i]##*/}"
core_name="${core_name#clean_}"
core_name="${core_name%_1.fastq}"
echo "${files[$i]} ${files[$i+1]} ${out_dir}output_${core_name}_pe.sam
```

```
${out_dir}summary_of_${core_name}_pe.txt"
done | parallel -j ${n_files} --colsep ' ' run_hisat2
```

A.5. Automate STAR (Single-End)

Ref: `src/auto_star_single.sh`

```
#!/usr/bin/env bash

# Runs STAR single_end in parallel for each read, hardcoded code.
# This script expects two arguments, one that is the path in which the
# files will be search on and the second where output files will be generated
# Usage:
#   ./auto_star_single.sh <input_dir> <output_dir>
#   ./auto_star_single.sh ../data/trimmed_fastqs ../results/star/single_end

# Same principle as `auto_fastp.sh` and `auto_hisat2.sh` and
`auto_hisat2_pe.sh`
# Create an array of the names that match: clean_*_1.fastq
mapfile -t files < <(find "$1" -maxdepth 1 -type f -name "clean_*_1.fastq" |
sort)

# Count how many read_files are
n_files=$(( ${#files[@]} ))

# Hardcoded function
run_star() {
    # Arguments
    #   --runMode alignReads: Set STAR to alignment mode
    #   --genomeDir: Path of the STAR index of the reference genome
    #   --readFilesIn: Input of the fastq file of single_end reads, taken from
the first argument passed to the function ($1)
    #   --outSAMtype SAM: Output alignment file in SAM format
    #   --outFileNamePrefix: Prefix for STAR output files ($2)
    #   --outSAMunmapped None: Don't include unmapped reads in SAM output,
similar to --no-unal in hisat2
    #   --runThreadN: Number of threads for STAR execution
    #   --outFilterMultimapNmax: Maximum number of multiple alignments allowed
per read, similar to -k in hisat2
    STAR --runMode alignReads \
         --genomeDir
/export/space3/users/silvanac/transcriptomica_2026/indexes/mm39.gencode.M36.st
ar/ \
         --readFilesIn "$1" \
         --outSAMtype SAM \
```

```
        --outFileNamePrefix "$2" \
        --outSAMunmapped None \
        --runThreadN 2 \
        --outFilterMultimapNmax 7
}
# Export the function, needed for parallel to use it
export -f run_star

# Take the second argument passed to the main call ($2) and remove a "/"
adding at the end a "/"
out_dir="${2%/}/"
# For each read_file do ... until i is less than the number of elements in the
array "files"
# starting with i = 0
for (( i=0; i<${#files[@]}; i+=1 )); do
# Build the core_name var, it takes the element "i" of the array "files"
# starting from the front of the string will eliminate all "*" until a "/" is
found,
# then again starting from the front will eliminate the first instance of
"clean_",
# finally starting from the back of the string will eliminate the ".fastq"
# e.g:
#   ../../../data/trimmed_fastqs/clean_SRR1111_1.fastq --> SRR1111_1
core_name="${files[$i]##*/}"
core_name="${core_name#clean_}"
core_name="${core_name%.fastq}"
# The echo will send 2 strings:
#   1st -> The fastq file of single-end reads
#   2nd -> STAR output prefix path
# These strings are build in the echo
echo "${files[$i]} ${out_dir}output_${core_name}_"
done | parallel -j ${n_files} --colsep ' ' run_star
```

A.6. Automate STAR (Paired-End)

Ref: `src/auto_star_pe.sh`

```
#!/usr/bin/env bash

# Runs STAR paired_end in parallel for each read, hardcoded code.
# This script expects two arguments, one that is the path in which the
# files will be search on and the second where output files will be generated
# Usage:
#   ./auto_star_pe.sh <input_dir> <output_dir>
#   ./auto_star_pe.sh ../data/trimmed_fastqs ../results/star/paired_end
```

```bash
# Same principle as `auto_fastp.sh` and `auto_hisat2.sh` and
`auto_hisat2_pe.sh` and `auto_star_single.sh`
# Create an array of the names that match: clean_*.fastq
mapfile -t files < <(find "$1" -maxdepth 1 -type f -name "clean_*.fastq" |
sort)

# Count how many read_files are / 2, pe reads
n_files=$(( ${#files[@]} / 2 ))

# Hardcoded function
run_star() {
    # Arguments
    #   --runMode alignReads: Set STAR to alignment mode
    #   --genomeDir: Path of the STAR index of the reference genome
    #   --readFilesIn: Input of paired-end reads, taken from the first and
second arguments passed to the function ($1 and $2)
    #   --outSAMtype SAM: Output alignment file in SAM format
    #   --outFileNamePrefix: Prefix for STAR output files ($3)
    #   --outSAMunmapped None: Don't include unmapped reads in SAM output,
similar to --no-unal in hisat2
    #   --runThreadN: Number of threads for STAR execution
    #   --outFilterMultimapNmax: Maximum number of multiple alignments allowed
per read, similar to -k in hisat2
    #   --outSAMattributes NH HI AS NM: Include key alignment tags in the SAM
output
    STAR --runMode alignReads \
         --genomeDir
/export/space3/users/silvanac/transcriptomica_2026/indexes/mm39.gencode.M36.st
ar/ \
         --readFilesIn "$1" "$2" \
         --outSAMtype SAM \
         --outFileNamePrefix "$3" \
         --outSAMunmapped None \
         --runThreadN 2 \
         --outFilterMultimapNmax 7 \
         --outSAMattributes NH HI AS NM
}
# Export the function, needed for parallel to use it
export -f run_star

# Take the second argument passed to the main call ($2) and remove a "/"
adding at the end a "/"
out_dir="${2%/}/"
# For each read_file do ... until i is less than the number of elements in the
array "files"
```

```
# starting with i = 0, advancing by 2 because each sample has _1 and _2
for (( i=0; i<${#files[@]}; i+=2 )); do
# Build the core_name var, it takes the element "i" of the array "files"
# starting from the front of the string will eliminate all "*" until a "/" is
found,
# then again starting from the front will eliminate the first instance of
"clean_",
# finally starting from the back of the string will eliminate the "_1.fastq"
# e.g:
#   ../../../data/trimmed_fastqs/clean_SRR1111_1.fastq --> SRR1111
core_name="${files[$i]##*/}"
core_name="${core_name#clean_}"
core_name="${core_name%_1.fastq}"
# The echo will send 3 strings:
#   1st -> The fastq file of forward reads
#   2nd -> The fastq file of reverse reads
#   3rd -> STAR output prefix path
# These strings are build in the echo
echo "${files[$i]} ${files[$i+1]} ${out_dir}output_${core_name}_pe_"
done | parallel -j ${n_files} --colsep ' ' run_star
```

B.1. Normalización de Estadísticas con Rust

Ref: `src/stats/src/lib.rs` y `main.rs`. Se creó una herramienta en Rust especializada que genera un resumen consolidado de estadísticas en formato similar a fasta.
`main.rs`:

```
use std::path::Path;
use std::{env, process};
use stats::{collect_hisat2, collect_star, Entry, Config};

// Creates a normalized "fasta-like" stats for "HISAT2" and "STAR"
// Usage:
//      ./stats <results_dir>
//      ./stats results/

fn main() {
    // Build the parser
    let config: Config = Config::build(env::args()).unwrap_or_else(|err| {
        eprint!("[ERROR] While parsing arguments {err}, missing directory");
        process::exit(1);
    });
    // Path of the results where hisat2 and star subfolder exists
    let results_dir = Path::new(&config.results_path);
```

```rust
    // Check if the results directory exists, else stop the execution
    if !results_dir.exists() {
        eprintln!("[ERROR] Directory not found");
        process::exit(1);
    }

    // Create the vector of all Entry structs
    let mut all: Vec<Entry> = Vec::new();

    // Call to a function to generate the vector of Entry of hisat2
    all.extend(collect_hisat2(results_dir));

    // Call to a function to generate the vector of Entry of star
    all.extend(collect_star(results_dir));

    // Print the Entry structs results
    for e in &all {
        println!(">ASSEMBLER: {} READ: {} ({})", e.assembler, e.srr, e.mode);
        println!("  Overall alignment: {}%", e.overall);
        println!("\tNon aligned: {}%", e.non_aligned);
        println!("\tAligned only once: {}%", e.once);
        println!("\tAligned >1 times: {}%", e.multi);
    }
}
```

`lib.rs`:

```rust
// Use module "filesystem", for manipulation of files
// Use struct "Path" of the module path, for inspecting system paths
use std::fs;
use std::path::Path;


// It also uses the method "new" of the struct "Regex" in the module "regex"

// Returns four owned String types
// Takes one argument: A reference to a string slice
// Parses the summary of hisat2
fn parse_hisat2_summary(content: &str) -> (String, String, String, String) {
    // Searchs pattern for any number until the percentaje before "overall
alignment rate"
    let regex_overall_alignment_rate = regex::Regex::new(r"([\d.]+)% overall
alignment rate").unwrap();

    // For paired-end summary files
    // The regex pattern will look for any number until a "%" before <String>
```

```
that matches
    let regex_aligned_zero_pe = regex::Regex::new(r"([\d.]+)%\) aligned
concordantly 0 times").unwrap();
    let regex_aligned_once_pe = regex::Regex::new(r"([\d.]+)%\) aligned
concordantly exactly 1 time").unwrap();
    let regex_multi_aligned_pe = regex::Regex::new(r"([\d.]+)%\) aligned
concordantly >1 times").unwrap();

    // For single-end
    // Same principle as single-end
    let regex_align_zero = regex::Regex::new(r"([\d.]+)%\) aligned 0
times").unwrap();
    let regex_align_once = regex::Regex::new(r"([\d.]+)%\) aligned exactly 1
time").unwrap();
    let regex_align_multi = regex::Regex::new(r"([\d.]+)%\) aligned >1
times").unwrap();

    // Creating the own types with the number gathered from previous
expressions

    // For the overal aligment
    let overall: String = regex_overall_alignment_rate // Pass the pattern
        .captures(content) // Search the pattern in the content
        .map(|c| c[1].to_string()) // Take the first group "([\d.]+)" of the
match
        .unwrap_or("N/A".into()); // Get the number or if not found return N/A
as String

    let zero: String = regex_aligned_zero_pe
        .captures(content)
        .or_else(|| regex_align_zero.captures(content))
        .map(|c| c[1].to_string())
        .unwrap_or("N/A".into());

    let once: String = regex_aligned_once_pe
        .captures(content)
        .or_else(|| regex_align_once.captures(content))
        .map(|c| c[1].to_string())
        .unwrap_or("N/A".into());

    let multi: String = regex_multi_aligned_pe
        .captures(content)
        .or_else(|| regex_align_multi.captures(content))
        .map(|c| c[1].to_string())
        .unwrap_or("N/A".into());
```

```rust
    // Return the 4 String
    (overall, zero, once, multi)
}

// Returns four owned String types
// Takes one argument: A reference to a string slice
// Parses the summary of star
fn parse_star_summary(content: &str) -> (String, String, String, String) {
    // Regex pattern that will search any digits after <String> and group with
parenthesis
    let regex_unique_mapped = regex::Regex::new(r"Uniquely mapped reads
%\s*\|\s*([\d.]+)%").unwrap();
    let regex_multi_mapped = regex::Regex::new(r"% of reads mapped to multiple
loci\s*\|\s*([\d.]+)%").unwrap();
    let regex_toomany_mapped = regex::Regex::new(r"% of reads mapped to too
many loci\s*\|\s*([\d.]+)%").unwrap();

    // Gathering the numbers of the stats

    let unique: f64 = regex_unique_mapped // Use this pattern
        .captures(content) // Captures the match in the "content"
        .map(|c| c[1].parse().unwrap_or(0.0)) // Map to an Option<f64>
        .unwrap_or(0.0); // Return the number if found, or return 0.0 if not
found

    let multi: f64 = regex_multi_mapped
        .captures(content)
        .map(|c| c[1].parse().unwrap_or(0.0))
        .unwrap_or(0.0);

    let toomany: f64 = regex_toomany_mapped
        .captures(content)
        .map(|c| c[1].parse().unwrap_or(0.0))
        .unwrap_or(0.0);

    // Add up all the aligns
    let overall = unique + multi + toomany;
    // Calculate the non aligned
    let non_aligned = 100.0 - overall;

    // Convert to String type with 2 decimals using the macro format!
    (
        format!("{:.2}", overall),
        format!("{:.2}", non_aligned),
        format!("{:.2}", unique),
        format!("{:.2}", multi + toomany),
```

```
    )
}

//Creating the struct that manages the output of the "fasta-like"
pub struct Entry {
    pub assembler: String,
    pub srr: String,
    pub mode: String,
    pub overall: String,
    pub non_aligned: String,
    pub once: String,
    pub multi: String,
}

// Collect the files of hisat2 summaries
// Returns: A vector of Entry structs
pub fn collect_hisat2(results_dir: &Path) -> Vec<Entry> {
    // Arguments:
    //      (results_dir: &Path): A reference to a Path struct

    // Create the Regex pattern, to search for the files
    let regex_pattern_file =
regex::Regex::new(r"summary_of_(SRR\d+)_(pe|1)\.txt").unwrap();
    // Initialize the Vector of Entry
    let mut entries: Vec<Entry> = Vec::new();

    // Search for the subdirectories in hisat2
    for subdir in &["paired_end", "single_end"] {
        let dir = results_dir.join("hisat2").join(subdir);
        // If the directory does not exists search for the next one
        if !dir.exists() {
            continue;
        }

        // Create the files variable, that contains a Vector of PathBuf
        let mut files: Vec<_> = fs::read_dir(&dir) // Reads the directory
            .unwrap() // Unwraps the result
            .filter_map(|e| e.ok()) // Checks if the Result enum is ok and
discards the None variants
            .map(|e| e.path()) // Gathers the Path of the remaining files
            .filter(|p| { // Filter to keep only Paths
                p.file_name() // Gathers the file name
                    .and_then(|n| n.to_str()) // Calls .to_str() if Some(T)
case
                    .map(|n| n.starts_with("summary_of_")) // Maps the
Option<&str> to Option<bool>
```

```
                    .unwrap_or(false) // Keep those that passed the filter
            })
            .collect(); // Create the Vec<PathBuf>
        files.sort(); // Sort the files

        // For each file path in the Vector of File paths
        for fpath in files {
            // Gathers the file name given the file path
            let fname = fpath.file_name()
                .unwrap()
                .to_str()
                .unwrap();

            // If the regex capture is the Some(caps) variant
            if let Some(caps) = regex_pattern_file.captures(fname) {
                // Selects the first group of the regex_pattern_file ->
"SRR11111"
                let srr = caps[1].to_string();
                // Selects the second group of the regex_pattern_file ->
"pe|1"
                // and then uses "PE" or "SE"
                let mode = if &caps[2] == "pe" { "PE" } else { "SE" };
                // Reads the file to a string
                let content = fs::read_to_string(&fpath).unwrap();
                // Call to the function
                let (overall, non_aligned, once, multi) =
parse_hisat2_summary(&content);
                // Pushing the Entry structs for the HISAT2
                entries.push(Entry {
                    assembler: "HISAT2".into(), // Assembler name
                    srr, // Pass the srr match
                    mode: mode.into(), // Pass the mode into a String
                    overall, // Pass the stats gather from the function
                    non_aligned,
                    once,
                    multi,
                });
            }
        }
    }
    entries
}

// Same priciple, but with the expression of Star summary files
pub fn collect_star(results_dir: &Path) -> Vec<Entry> {
    // Pattern to search for the file names
```

```
    let re_file =
regex::Regex::new(r"output_(SRR\d+)_(pe|1)_Log\.final\.out").unwrap();
    // Initialize the Vector of Entry
    let mut entries = Vec::new();

    // Search in the subdirectory
    for subdir in &["paired_end", "single_end"] {
        // Creates the sudirectory path, given the results directory
        let dir = results_dir.join("star").join(subdir);

        // If those subdirectories are not found pass to next
        if !dir.exists() {
            continue;
        }
        // Creates the variable files
        // It will contain a Vector of PathBuf
        let mut files: Vec<_> = fs::read_dir(&dir)
            .unwrap()
            .filter_map(|e| e.ok())
            .map(|e| e.path())
            .filter(|p| {
                p.file_name()
                    .and_then(|n| n.to_str())
                    .map(|n| n.ends_with("_Log.final.out"))
                    .unwrap_or(false)
            })
            .collect();
        files.sort();

        // Process each file to gather the stats
        for fpath in files {
            let fname = fpath.file_name().unwrap().to_str().unwrap();
            if let Some(caps) = re_file.captures(fname) {
                let srr = caps[1].to_string();
                let mode = if &caps[2] == "pe" { "PE" } else { "SE" };
                let content = fs::read_to_string(&fpath).unwrap();
                let (overall, non_aligned, once, multi) =
parse_star_summary(&content);
                // Push the Entry struct
                entries.push(Entry {
                    assembler: "STAR".into(),
                    srr,
                    mode: mode.into(),
                    overall,
                    non_aligned,
                    once,
```

```
                    multi,
                });
            }
        }
    }
    // Return the entries vec
    entries
}

// Struct to manage parse of arguments
pub struct Config {
    pub results_path: String,
}

// Implementation block for the Config struct
impl Config {
    // Function to build a Config struct
    pub fn build(
        // Take mutability of the argument, acepts an data type that
implements Iterator with elements of String
        mut args: impl Iterator<Item = String>,
        // Returns a Result enum, Ok(Config) or Err(&'static str)
    ) -> Result<Config, &'static str> {
        // Pass the first element that is path of the executable
        args.next();

        let results_path = match args.next() {
            Some(arg) => arg,
            None => return Err("Didn't get a directory path")
        };

        Ok(Config { results_path })
    }
}
```

Nota: Ejecutable en src/stats_rs

B.2. Graficación

Ref: `results/plot_stats.R`

```
# Load libraries
library(ggplot2)
library(stringr)

# Read the results file
```

```r
path <- "results/stats/final_stats.txt"
data_text <- paste(readLines(path), collapse = "\n")

# Parse text lines
lines <- readLines(path)

# Initialize all the variables
assemblers <- c()
srrs <- c()
types <- c()
overall <- c()
non_aligned <- c()
once <- c()
multi <- c()

for (i in seq_along(lines)) {
  # Parse the "fasta-like" file to the previous variables
  if (grepl("^>ASSEMBLER:", lines[i])) {
    # Assign them trough REGEX patterns
    assemblers <- c(assemblers, str_extract(lines[i], "(?<=ASSEMBLER: )\\w+"))
    srrs <- c(srrs, str_extract(lines[i], "SRR\\d+"))
    types <- c(types, str_extract(lines[i], "(?<=\\()PE|SE(?=\\))"))
    overall <- c(overall, as.numeric(str_extract(lines[i + 1], "[\\d.]+(?
=%)")))
    non_aligned <- c(
      non_aligned,
      as.numeric(str_extract(lines[i + 2], "[\\d.]+(?=%)"))
    )
    once <- c(once, as.numeric(str_extract(lines[i + 3], "[\\d.]+(?=%)")))
    multi <- c(multi, as.numeric(str_extract(lines[i + 4], "[\\d.]+")))
  }
}

# Create a data.frame
df_wide <- data.frame(
  Assembler = assemblers,
  SRR = srrs,
  Type = types,
  Overall = overall,
  Non_Aligned = non_aligned,
  Once = once,
  Multi = multi
)

# Transform the data.frame to a long format, easier to manipulate
# It expands rows to columns to build a longer data.frame
```

```r
df_long <- tidyr::pivot_longer(
  df_wide,
  cols = c(Overall, Non_Aligned, Once, Multi),
  names_to = "Metric",
  values_to = "Percentage"
)

# Create a new column, with the Assembler and Type in the same field
df_long$Group <- paste(df_long$Assembler, df_long$Type, sep = " - ")

# Tags for the metrics
df_long$Metric <- factor(
  df_long$Metric,
  levels = c("Overall", "Non_Aligned", "Once", "Multi"),
  labels = c(
    "Overall Alignment",
    "Non Aligned",
    "Aligned Once",
    "Aligned >1 times"
  )
)

# Geom point plot, showing of stat with differece in color and shape
point_plot <- ggplot(
  df_long,
  aes(x = SRR, y = Percentage, color = Group, shape = Metric)
) +
  geom_point(size = 3.5, alpha = 0.7) +
  theme_minimal() +
  labs(
    title = "Alignment Statistics per SRR",
    subtitle = "All metrics by Assembler and Read Type (PE vs SE)",
    x = "SRR Accession",
    y = "Percentage (%)",
    color = "Condition",
    shape = "Metric"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_y_continuous(limits = c(0, 100))

# Save in high quality the plot
ggsave(
  "results/plots/stats_points.png",
  plot = point_plot,
  dpi = 800,
  width = 11.25,
```

```
  height = 7.5,
  bg = "white"
)
```